

Projektbericht — CocktailPlaner

Hochschule: Martin-Luther-Universität Halle-Wittenberg
Veranstaltung: Webtechnologien (Semester-Projekt)
Präsentation: 16. Juli 2026, 12–16 Uhr, Pool 3.02
Autor: Henry Schach
Projektname: Cocktailrezepte / CocktailPlaner

1. Projektbeschreibung

Der **CocktailPlaner** ist eine browserbasierte Single-Page-Application (SPA), mit der Nutzer Events (z. B. Cocktailpartys) planen können. Die App berechnet automatisch, wie viele Flaschen und Zutaten für eine bestimmte Gästezahl benötigt werden, und erstellt eine druckfertige Einkaufsliste.

Ergänzend bietet die App ein vollständiges **Cocktail-Glossar** mit Suchfunktion, das Rezepte, Zutaten und ein proportionales SVG-Glas für jeden Cocktail anzeigt.

2. Erfüllte Mindestvoraussetzungen

Anforderung	Erfüllt	Details
HTML	✓	Semantisches HTML5, <code>public/index.html</code> als SPA-Shell
CSS	✓	Bulma 0.9.4 (CSS-Framework) + eigene Klassen in <code>public/style.css</code> ; responsive auf allen Viewport-Breiten
SVG	✓	Proportionaler Cocktail-Messbecher (<code>src/View/CocktailSvg.elm</code>) — Zutaten als farbige Schichten, programmatisch aus Elm generiert
Elm	✓	Vollständig in Elm 0.19.1; <code>Browser.application</code> ; keine JavaScript-Logik
HTTP-Anfragen	✓	4 API-Endpunkte von TheCocktailDB: Kategorien, Cocktailliste, Vollrezept, Namenssuche
URL-Navigation	✓	3 Routen (<code>/</code> , <code>/einkaufsliste</code> , <code>/glossar</code>), History API via <code>Browser.Navigation</code>

3. Technologie-Stack

Technologie	Version	Zweck
Elm	0.19.1	Gesamte App-Logik (kein JavaScript)
Bulma	0.9.4	CSS-Framework (Grid, Navbar, Cards, Modal)
Font Awesome	6.5.0	Icons (Lupe, Speichern, Laden, ...)
TheCocktailDB	v1 (free)	Externe HTTP-API: Rezepte, Zutaten, Thumbnails
elm/http	2.0.0	HTTP-Client in Elm
elm/svg	1.0.1	SVG-Cocktailglas

elm/url	1.0.0	URL-Parsing für Routing
---------	-------	-------------------------

4. Architektur

Die App folgt der Elm Architecture (Model–Update–View):

```
src/
├─ Main.elm          – Browser.application, init/update/view-Wiring
├─ Types.elm         – Alle Typen (Model, Msg, Route, Event, Ingredient ...)
├─ Route.elm         – URL-Parser, fromUrl / toPath
├─ Api.elm           – HTTP-Funktionen + JSON-Decoder + Mengenberechnung
├─ Codec.elm         – JSON-Encode/Decode für Save/Load (localStorage-Ersatz)
└─ View/
   ├─ Topbar.elm     – Navigationsleiste (responsiv, Burger-Menü)
   ├─ Home.elm       – Event-Verwaltung + Cocktail-Suche pro Event
   ├─ Shopping.elm    – Einkaufsliste mit Packungsgrößen-Dropdown
   ├─ Glossar.elm     – 3-Spalten-Modal: Kategorien / Cocktailliste / Detail
   └─ CocktailSvg.elm – SVG-Messbecher (proportionale Zutaten-Schichten)
```

Routing: `Browser.application` fängt alle Link-Klicks ab; `Route.fromUrl` mappt auf `HomeRoute` | `ShoppingRoute` | `GlossarRoute`. URL-Änderungen werden mit `Nav.pushUrl` ausgelöst.

HTTP-Daten: Werden lazy geladen (erst beim ersten Öffnen des Glossars) und in `Dict String FullCocktail` gecacht, damit wiederholte Klicks keinen neuen Request erzeugen.

Persistenz: Kein Server, kein LocalStorage — der Nutzer kann den Projektstand als JSON-String kopieren (`Speichern`) und später wieder einfügen (`Laden`). Format ist stabil und versioniert via `Codec.elm`.

5. Funktionen

Home-Seite

- Events anlegen (Name + Gästeanzahl) mit Validierung
- Events duplizieren, löschen, inline umbenennen
- Pro Event: Cocktails per Freitextsuche finden (`TheCocktailDB search.php`)
- Portionen pro Cocktail einstellen (± Buttons)
- Direkt zur Einkaufsliste springen

Einkaufsliste

- Aggregiert alle Zutaten aller Cocktails × Portionen × Gästeanzahl
- Einheitenkonvertierung: oz, cl, tsp, tbs, cup, shot, dash, drop → ml
- Pro Zutat: Packungsgröße wählen (250 ml / 500 ml / 700 ml / 1000 ml / 1500 ml)
- Zeigt an: Gesamtmenge (ml), benötigte Packungen (aufgerundet), Flaschen insgesamt

Cocktail-Glossar

- Volltext-Namenssuche (ab 2 Zeichen, `search.php`)
- Browsing nach Kategorien (Cocktail, Shot, Beer, ...)
- 3-Spalten-Layout: Kategorien | Cocktailliste | Detail
- Detail-Ansicht: Foto + SVG-Glas (Zutaten proportional als Farbschichten), Zutaten-Tags, Rezept
- Responsive: Spalten 1+2 auf Tablets ausgeblendet, Stacked-Layout auf Smartphones

Speichern & Laden

- JSON-Export des gesamten Projektstands (Events, Portionen, Packungsgrößen)
- Warnung beim Laden wenn bereits ein Projekt geöffnet ist

6. Interessantes Detail: Mengen-Konvertierung in Elm

TheCocktailDB liefert Mengenangaben als Rohtext: "1 1/2 oz", "2 cl", "3 dashes", "Garnish". Die App konvertiert diese beim Dekodieren in einen strukturierten Typ:

```
type IngredientAmount
  = LiquidMl Float -- in Milliliter, z. B. 44.355 ml (= 1.5 oz)
  | PieceCount Float -- Stückzahl, z. B. 3.0 (Limetten)
  | UnknownAmount -- Garnish, "Top", nicht parsbar
```

Der Parser `parseMeasureToAmount` in `Api.elm` erkennt gemischte Zahlen (1 1/2), Brüche (3/4), Bereiche (6-8 → Untergrenze), und alle gängigen Einheiten. Nur `LiquidMl` -Mengen erscheinen in der Einkaufsliste — Garnishes und unbekannte Mengen werden separat angezeigt.

7. Projektverlauf

Datum	Meilenstein
26.06.2026	Toolchain, ADRs 1–5, Prototyp (alle 5 Technologien validiert)
29.06.2026	ADRs 6–9, Multi-Modul-Struktur, Glossar mit API, SVG-Glas, Events
30.06.2026	Shopping-View, Save/Load, Bugfixes (13 gefundene Bugs), Glossar-Suche

8. Offene Punkte / mögliche Erweiterungen

- Deployment als Live-Demo (GitHub Pages o. ä.)
- Druckansicht für die Einkaufsliste
- Lokale Favoriten-Liste

Quellcode: <https://github.com/0x520Uni/CocktailPlanner>

Live-Demo: <https://0x520uni.github.io/CocktailPlanner/>